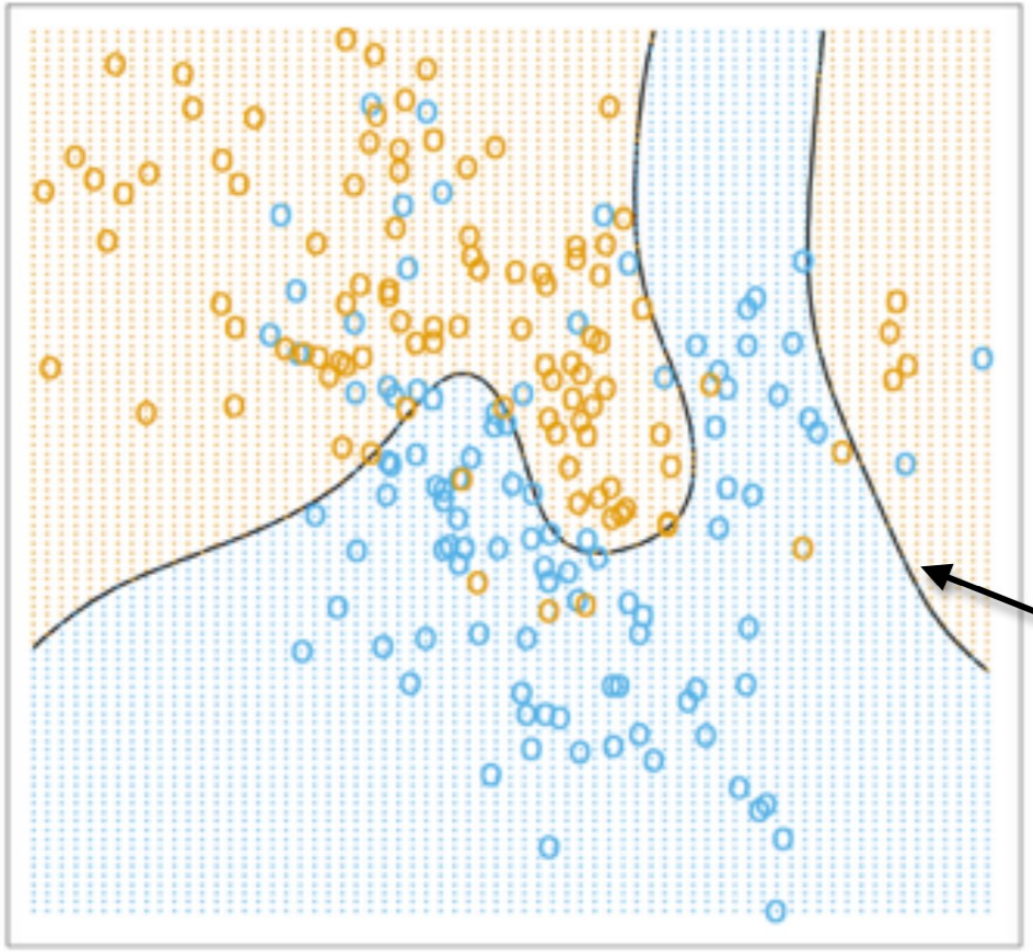
Lecture 12: Local Methods & Kernels

Nearest neighbor methods

- Imagine that you have points on an axis, and you want to categorize a new point (a piece of test data)
  - One strategy could just be: what data point are you closest to? Well, then, you have the same category as that point

Classification data: Bayes classifier



Training data:

True label: +1

True label: -1

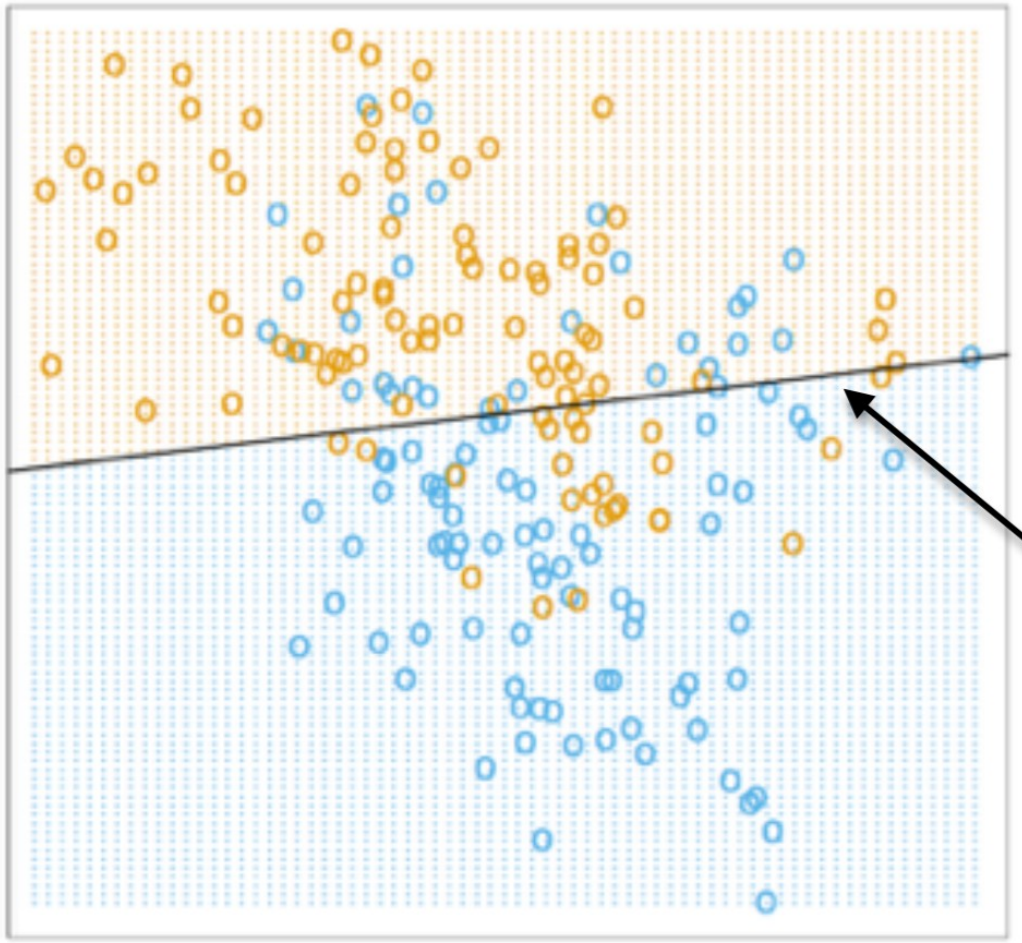
Optimal “Bayes” classifier:

$$\mathbb{P}(Y = 1|X = x) = \frac{1}{2}$$

Predicted label: +1

Predicted label: -1

Classification data: linear decision boundary



Training data:

True label: +1

True label: -1

Learned:

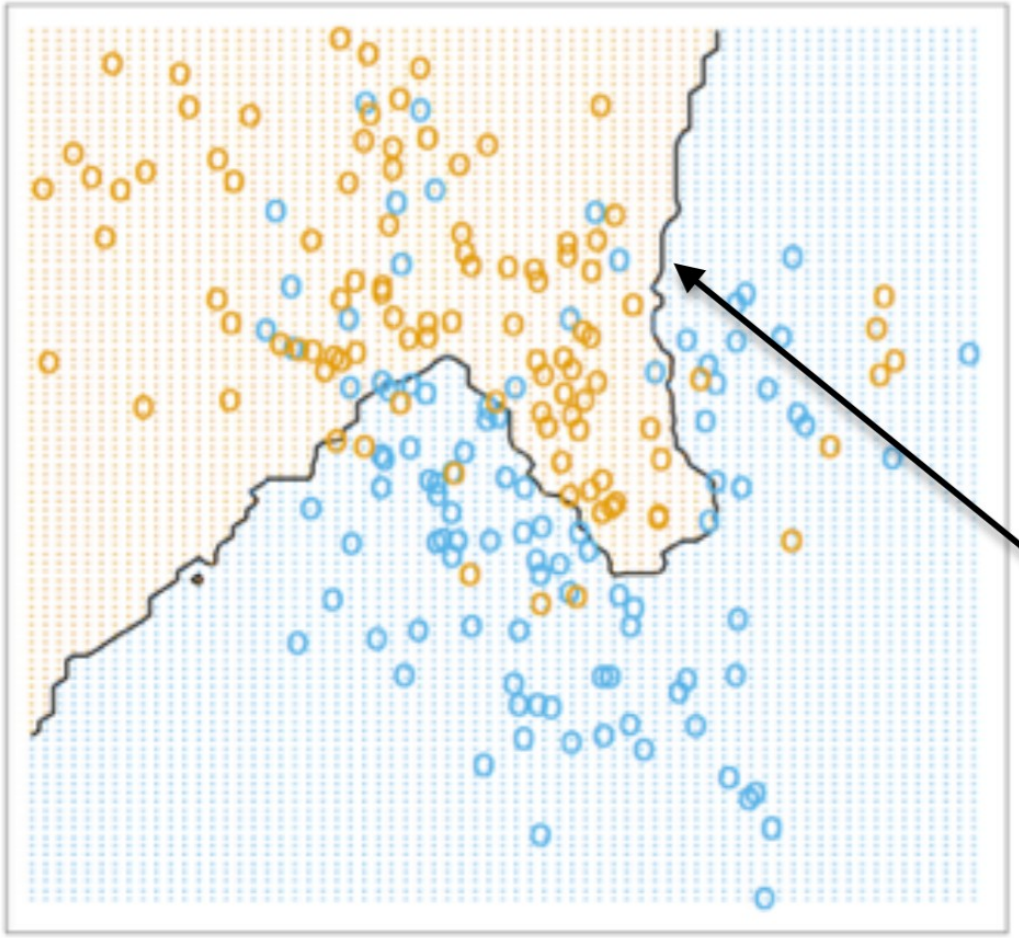
Linear Decision boundary

$$x^T w + b = 0$$

Predicted label: +1

Predicted label: -1

15 nearest neighbor boundary



Training data:

True label: +1

True label: -1

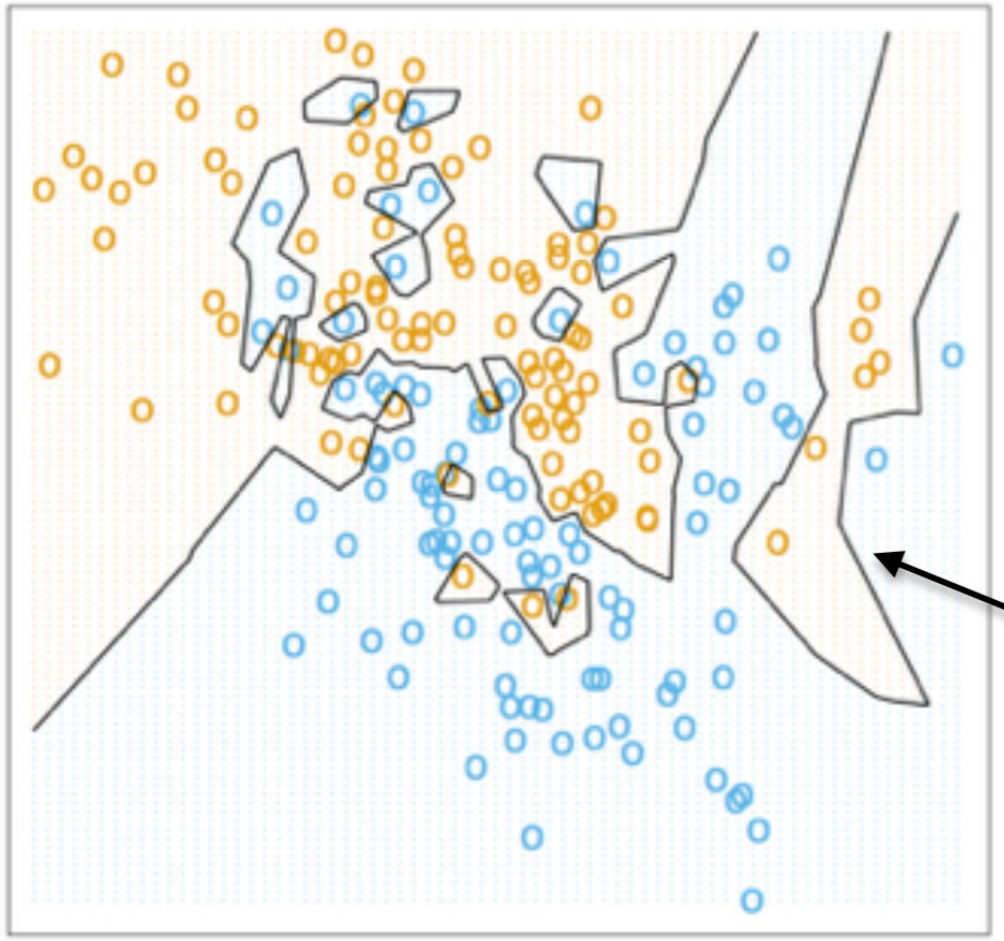
Learned:

15 nearest neighbor decision boundary (majority vote)

Predicted label: +1

Predicted label: -1

1 nearest neighbor boundary



Training data:

True label: +1

True label: -1

Learned:

1 nearest neighbor decision boundary (majority vote)

Predicted label: +1

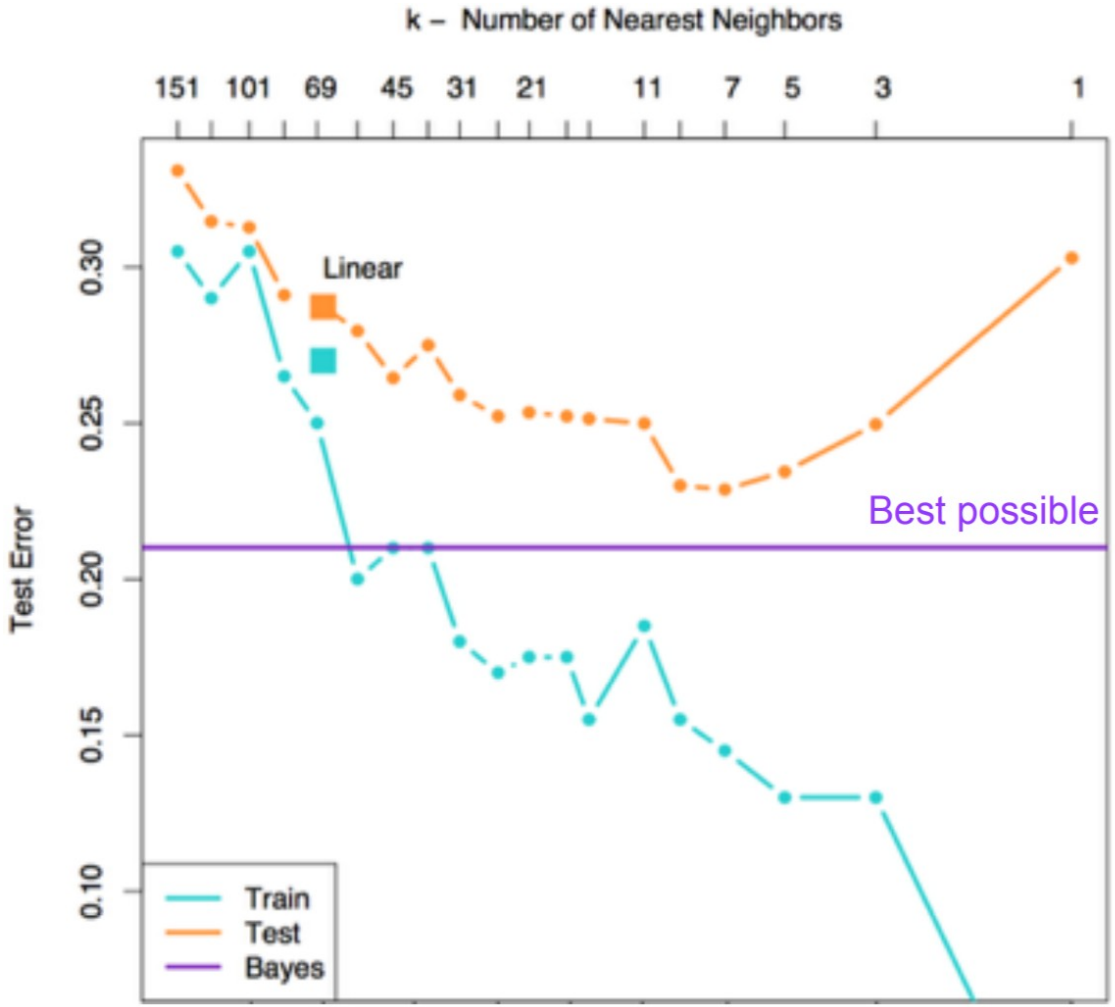
Predicted label: -1

Nearest neighbor

- Nearest neighbor has tradeoffs (pros and cons)

- And a decision point: we looked at 15 vs 1 nearest neighbor—what's the difference and what's the "best" number to use (if such a thing exists)

k-nearest neighbor & error



Bias-Variance tradeoff

As k->infinity?

Bias:

Variance:

As k->1?

Bias:

Variance:

k-nearest neighbor & error

- As n->∞
  - Your data becomes *dense*
  - Your error decreases
  - Your error remains roughly twice the Bayes error

- Regardless of your n, method performance largely depends on how well your training data represents the overall data
  - Which is generally true of machine learning, but especially true here
  - Draw an example of when this is especially true

Nearest neighbor summary

- Very simple to explain and implement
- No training! *Can* be computationally demanding depending on dataset size and k
- Can use any form of distance (not just Euclidean, as we did)
- Smoothing with kernels can improve performance

- With a lot of data, “local methods” have strong, simple theoretical guarantees. With not a lot of data, neighbors aren’t local and methods suffer for it

Kernel methods

- We've seen them before (SVM) and we'll see them again (PCA)
- Motivation: as the number of features increases, computational difficulty of learning can skyrocket
- Problem in machine learning: Represent (often high-dimensional) data
- Problem, re-framed: Calculate a similarity measure in the feature space, then apply machine learning algorithm to this similarity measure

What exactly is a kernel?

- A function that takes as input vectors in the original space and returns vectors in the feature space. This function is called a kernel function.

- Can be thought of as modeling the difference between each individual pair of data and then modeling those differences rather than trying to learn how every feature interacts across all of the data

Common kernels (you don't need to know these)

- Polynomials of degree exactly d

$$K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v})^d$$

- Polynomials of degree up to d

$$K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v} + 1)^d$$

- Gaussian (squared exponential) kernel

$$K(\mathbf{u}, \mathbf{v}) = \exp\left(-\frac{||\mathbf{u} - \mathbf{v}||_2^2}{2\sigma^2}\right)$$

- Sigmoid

$$K(\mathbf{u}, \mathbf{v}) = \tanh(\eta \mathbf{u} \cdot \mathbf{v} + \nu)$$